

Taller 1

INTEGRANTES:

Victor Pointud, Ci: 30.124.022, Carnet: 20221110061

Juan Salima, Ci: 30.142.789 Carnet: 20221110517

Bernardo Lander, Ci 28.472.409, Carnet 20211110549

Caracas, 22 de enero de 2024

Parte I Organizaciones de Archivo

1. Dado el siguiente modelo relacional.

Artista (<u>cod_artista</u>, nombre_real, nombre_artistico, nacionalidad, sexo, fecha_nacimiento) cod_artista: 4 bytes nombre_real: 40 bytes (B+, h = 3) nombre_artistico: 40 bytes (B+, h = 3) nacionalidad: 30 bytes sexo: 1 byte (masculino, femenino) fecha_nacimiento: 10 bytes Número de registros: 10.000 Género (<u>cod_genero</u>, nombre, descripcion) cod_genero: 4 bytes nombre: 40 bytes (B+, h = 3) descripcion: 50 bytes Número de registros: 75 Tiene (<u>cod_artista</u>, <u>cod_genero</u>, fecha_inicio) cod_artista: 4 bytes cod_genero: 4 bytes fecha_inicio: 10 bytes Número de registros: 20.000	Interpreta (<u>cod_artista</u>, <u>cod_cancion</u>, tipo) cod_artista: 4 bytes cod_cancion: 4 bytes Tipo 10 bytes (original, cover) Número de registros: 1.500.000 Cancion (<u>cod_cancion</u>, cod_album, cod_genero, nombre_cancion, duracion, descripcion) cod_cancion: 4 bytes (1000 nombres de canciones diferentes aprox) cod_album: 4 bytes cod_genero: 4 bytes nombre_cancion: 40 bytes (B+, h = 3) duracion: 10 bytes (min 1 y, max 7) Número de registros: 500.000 Album (<u>cod_album</u>, nombre, descripcion, año, disquera) cod_album: 4 bytes nombre: 40 bytes (B+, h = 3) descripcion: 50 bytes año: 5 bytes (18 años diferentes aprox, min 2005 y max 2023) disquera: 40 bytes (150 disqueras diferentes aprox) Número de registros: 6.000
--	--

Se utiliza un byte adicional para marcador de borrado. El tamaño del Bloque es B=2048 bytes. La velocidad de transferencia por bloque es 0,17 mseg

CALCULE (4 ptos)

1. El tamaño del registro TR del archivo Artista en bytes y el factor de bloqueo fdB asumiendo una **organización no extendida**.

Artista: Archivo de datos

cod_artista	nombre_real	nombre_artístico	nacionalidad	sexo	fecha_nacimiento
4	40	40	30	1	10

Tabla 1. Tamaño en bytes de los datos.

Nro de registros (nR): 10.000 (dado)

Tamaño del bloque (B): 2048 bytes (dado)

Tamaño del registro (R): $4 + 40 + 40 + 30 + 1 + 10 + 1$ (byte de borrado) = **126 bytes**

Factor de bloqueo (fb): $fb = \text{piso}(B/R) = \text{piso}(2048/126) = 16$

Tamaño del archivo (TA): $TA = \text{techo}(nR/fb) = \text{techo}(10.000/16) = 625 \text{ bloques}$

2. Calcule el tiempo medio en buscar un registro por campo clave si la **organización** es en **montículo**.

La organización de tipo montículo almacena los registros en el orden en que llegan, lo que implica que la búsqueda se realiza de manera lineal. Teniendo en cuenta que el tiempo medio de leer o escribir páginas en disco es

$D = 0,17 \text{ mseg/bloque}$, obtenemos:

Caso promedio toma un tiempo medio de:

$$TM = (TA \times D) / 2 = (625 \times 0,17) / 2 = 53,125 \text{ ms} = \mathbf{0,53 \text{ s}}$$

3. Suponiendo que el archivo está ordenado por cod_artista, Calcule en tiempo medio en buscar un código dado.

Caso promedio toma un tiempo medio de:

$$TM = [\text{Log}_2(TA) \times D] / 2 = [\text{Log}_2(10.000) \times 0,17] / 2 = \mathbf{0,789 \text{ ms}}$$

4. Exprese la función Hash usando la técnica de división resto, si el Sistema Manejador de Base de datos usa un bucket de 8 bloques del SO y se requiere que el archivo deje un 10% de espacio libre.

El tamaño del bucket está dado por:

$$B = 2048 \text{ bytes/bloque} \times 8 \text{ bloques} = \mathbf{16,384 \text{ bytes}}$$

Luego, la cantidad de registros que caben en un bloque es el siguiente:

$$fb = \text{piso}(B/R) = \text{piso}(16,384/126) = \mathbf{130 \text{ registros}}$$

Así, como se desea dejar un 10% de espacio libre, el número de buckets disponibles (M) será el número primo mayor a

$$\text{techo}[(nR/fb) \times (1, 1)] = \text{techo}[(75/130) \times (1, 1)] = \mathbf{1 \text{ buckets}}$$

Al ser 1 numero primo dado el techo se sube a 2, por lo que serian 2 buckets en total

Es decir, $M = \mathbf{2 \text{ buckets}}$ disponibles. En consecuencia, la función Hash

$h: K \rightarrow B$ está dada por:

$$h(k) = k \bmod 2$$

ÍNDICES (6 ptos)

Se tiene el archivo Artistas del ejercicio anterior, con apuntador a bloque p = 9 bytes: Si se construye un índice primario de dos niveles sobre cod_artista (recuerde dejar 1 byte para borrado)

Artista: Archivo de índice

Nivel 1

cod_artista	puntador

Nivel 2

cod_artista	puntador

Tabla 2. Tamaño en bytes de los datos.

1. ¿Ambos niveles del índice pueden ser dispersos?

Si, ambos niveles del índice pueden ser dispersos al estructurarse sobre un índice primario, donde los registros ya están organizados según el campo DI. En el primer nivel del archivo del índice, las entradas apuntan al primer registro de cada bloque en el archivo de datos. El segundo nivel a su vez también puede ser disperso apuntando solamente a bloques seleccionados del primer nivel del archivo índice.

2. ¿Cuál es el factor de bloqueo del índice?

Tamaño del registro (R): $R = 4 + 9 + 1 = 14$ bytes

Tamaño del bloque (B): $B = 2048$ bytes

Factor de bloqueo (fb): $fb = 146$

3. ¿Cuántas entradas de datos hay por nivel?

Nivel 1: $TAi \text{ nivel } 1 = \text{techo}(TA/fb) = \text{techo}(625/146) = 4$

Nivel 2: $TAi \text{ nivel } 2 = \text{techo}(TA/fb) = \text{techo}(4/146) = 0,03$

4. ¿Cuántos accesos se requieren para hallar un registro dado el código?

Al momento de buscar sobre el archivo disperso de 2 niveles tenemos los siguientes accesos a memoria:

Nivel 2: $\text{Log}_2(TAi \text{ nivel } 2) = \text{Log}_2(0,03) = -5,059$

Dado que el Nivel 2 es negativo y el mínimo de accesos debe ser 1 entonces el nivel 2 debe ser 1 acceso.

Nivel 1: 1 acceso a disco ya que el nivel 2 apunta directamente al bloque.

Archivo de datos: 1 acceso a disco ya que el nivel 1 apunta directamente al bloque.

Total: 3 accesos a disco para hallar un registro dado el código.

Si se construye un índice agrupado sobre el campo nombre_artístico y se sabe que hay 2.000 nombres diferentes.

1. ¿De qué tipo puede ser el índice: denso, disperso o cualquiera de los dos?

Los índices agrupados pueden ser tanto densos como dispersos. Sin embargo, al tomar en cuenta que la entidad jugador en su campo nombre puede tener 1.000 nombres diferentes sería más recomendable utilizar un índice disperso ya que si se utilizara un índice denso se requerirían de 2000 registros en el archivo índice para cada nombre distinto, cual podría resultar en un acceso y gestión ineficiente. Al utilizar un índice disperso se minimiza la cantidad de registros en el archivo índice, apuntando solo al bloque específico donde comienzan el valor más cercano al valor buscado.

2. ¿Cómo serían las entradas del índice?

Si el índice fuera **denso**, se tendrían 2000 entradas con dos campos, uno que corresponde a cada valor único de la clave no primaria sobre la que se ha construido el índice (nombre) y el apuntador al primer bloque del archivo de datos asociado a la clave 'nombre'.

Si el índice fuera **disperso**, se tendrían menos entradas que el índice denso, cada una con dos campos, uno para el valor único de la clave no primaria (nombre) que inicia un bloque en particular y el apuntador al bloque en el archivo de datos donde comienzan los registros con ese nombre o valores cercanos a ese nombre.

3. Calcule el tamaño del índice

nombre_artístico	Apuntador
40	9

Tabla 3. Tamaño en bytes del registro índice.

- **Tamaño del registro (R):** $R = 40 + 9 + 1 = 50$ bytes
- **Tamaño del bloque (B):** $B = 2048$ bytes (dado)
- **Factor de bloqueo:** $fb = 40$

Para índice denso:

- **Tamaño del archivo (TA):**

$$TA = \text{techo}(nR/fb) = \text{techo}(2.000/40) = \mathbf{50 \text{ bloques}}$$

Para índice disperso:

No es posible calcular con precisión el tamaño del índice disperso en este contexto, ya que no se dispone de información detallada sobre cómo se distribuyen los 2.000 nombres diferentes entre los 10.000 registros totales. Para determinar el tamaño exacto de un índice disperso, necesitaríamos conocer la distribución específica de los nombres y cómo se agruparán en las entradas del índice. Sin embargo, podemos afirmar con seguridad que el tamaño del índice disperso sería menor que el del índice denso en este escenario.

Si se construye un índice B+ secundario sobre el nombre_artístico

1. Calcule la altura máxima del árbol

El orden de un árbol B + determina la máxima expansión que cada nodo puede tener, lo cual permite calcular la altura máxima del árbol. Al comprender esta altura máxima, podemos estimar el recorrido más largo desde la raíz hasta las hojas, donde se localizan los punteros a los registros de datos. Para calcular el orden del árbol aplicamos la fórmula.

$$m = \text{piso}[(B+K) / (P+K)] = \text{piso}[(2048+40) / (9+40)] = \text{piso}(2048/49) \\ m = \mathbf{42 \text{ nodos hijos máximos}}$$

Donde:

M = orden del árbol.

B = Tamaño del bloque = **2048 bytes** (dado en el enunciado)

K = tamaño de la clave nombre = **40 bytes** (dado en el enunciado)

P = tamaño del apuntador = **9 bytes** (dado en el enunciado)

La altura mínima y máxima de un árbol B+ depende de su orden y se calcula con las fórmulas:

Altura mínima:

$$\text{Techo}[\text{Logm}(n)] = \text{Techo}[\text{Log}_{42}(2.000)] = \mathbf{3 \text{ niveles}}$$

Altura máxima:

$$\text{Techo}[\text{Logm}/2(n)] = \text{Techo}[\text{Log}_{42}/2(2.000)] = \mathbf{3 \text{ niveles}}$$

Donde

m: orden del árbol.

n: número total de valores distintos de **k = 2.000** (dado en el enunciado).

- 2. Si hay 10 personas de nombre artístico Wilfrido indique cuántos accesos son necesarios para realizar una consulta sobre ese nombre, en el peor de los casos.**

Para calcular cuántos accesos son necesarios al consultar los registros que coincidan con el nombre "Wilfrido" sobre un índice B + secundario se debe evaluar en primer lugar el camino desde la raíz del árbol hasta llegar a los registros correspondientes. Ya que estamos considerando el peor de los casos se debe tomar la altura máxima del árbol que corresponde a 3, hasta este punto corresponde a 3 accesos a memoria (1 por cada nivel, raíz, intermedio y hojas con los apuntadores a los nombres).

Por ser organización de tipo secundaria los registros que tienen el mismo valor de clave (nombre) se enlazan secuencialmente en el nivel más bajo (hojas); si existen 10 registros con el nombre Wilfrido el peor de los casos implica que el Wilfrido que se está buscando esté en el último lugar, por lo cual se deberá recorrer los 9 registros restantes.

En total el costo de búsqueda corresponde a la altura más registros de la búsqueda (H+S), es decir, **13** accesos a memoria donde 3 accesos son para recorrer el árbol B + desde la raíz hasta la hoja y 10 accesos para recorrer cada uno de los 10 registros con el nombre "Wilfrido".

Parte 2 Procesamiento de Consultas

Nombre real, nacionalidad y sexo de todos los artistas que hayan interpretado una canción de nombre “El farolito” y pertenezcan a un álbum entre los años 2005 y 2010.

1. Dé el árbol canónico para la consulta. (1 pto)

La consulta en **SQL** corresponde:

```
SELECT A.nombre_real, A.nacionalidad, A.sexo
FROM Artista A, Interpreta I, Cancion C, Album L
WHERE A.cod_artista = I.cod_artista
AND L.cod_cancion = C.cod_cancion
AND C.nombre_cancion = 'El farolito'
AND C.cod_album = L.cod_album
AND A.año BETWEEN 2005 AND 2010;
```

La consulta en **Álgebra Relacional** corresponde a:

```
 $\Pi$  nombre_real, nacionalidad, sexo(
  (  $\sigma$  nombre_cancion = 'El farolito'(Cancion))
   $\bowtie$  cod_album = cod_album
  ( $\sigma$  año  $\geq$  2005  $\wedge$  año  $\leq$  2010(Artista))
  )
   $\bowtie$  cod_cancion = cod_cancion
  Interpreta
   $\bowtie$  cod_artista = cod_artista
  Artista
  )
```

En consecuencia, el árbol canónico es el siguiente:

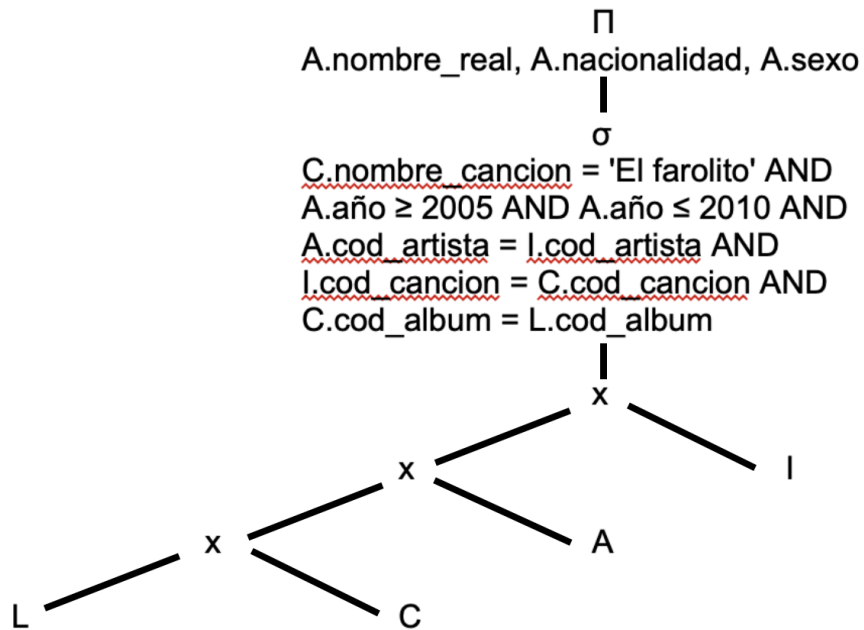
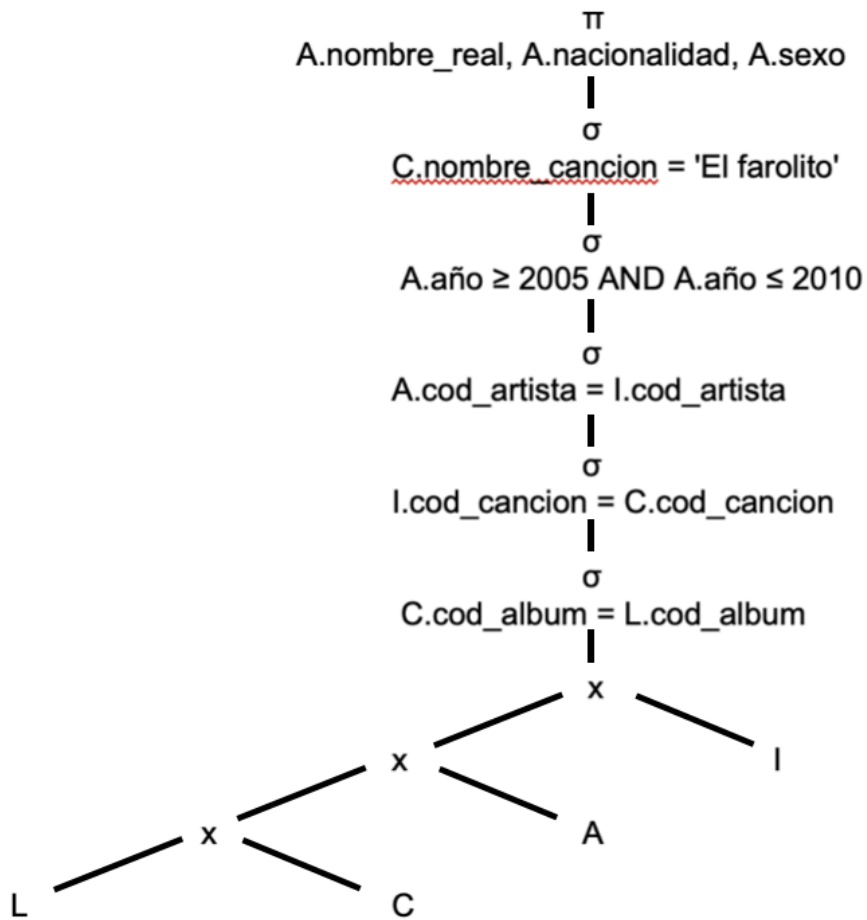


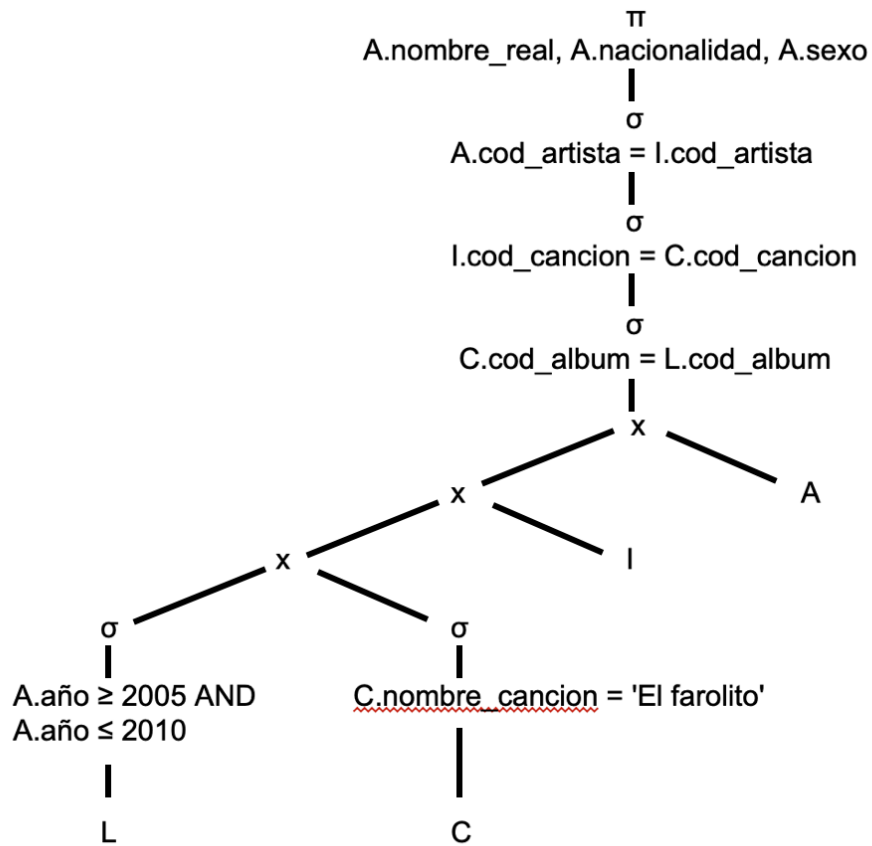
Imagen 1. Árbol Canónico de la consulta SQL.

2. Dé la secuencia de equivalencias (axiomas) que es necesario aplicar para transformar el árbol canónico en su equivalente optimizado heurísticamente. (4 pts)

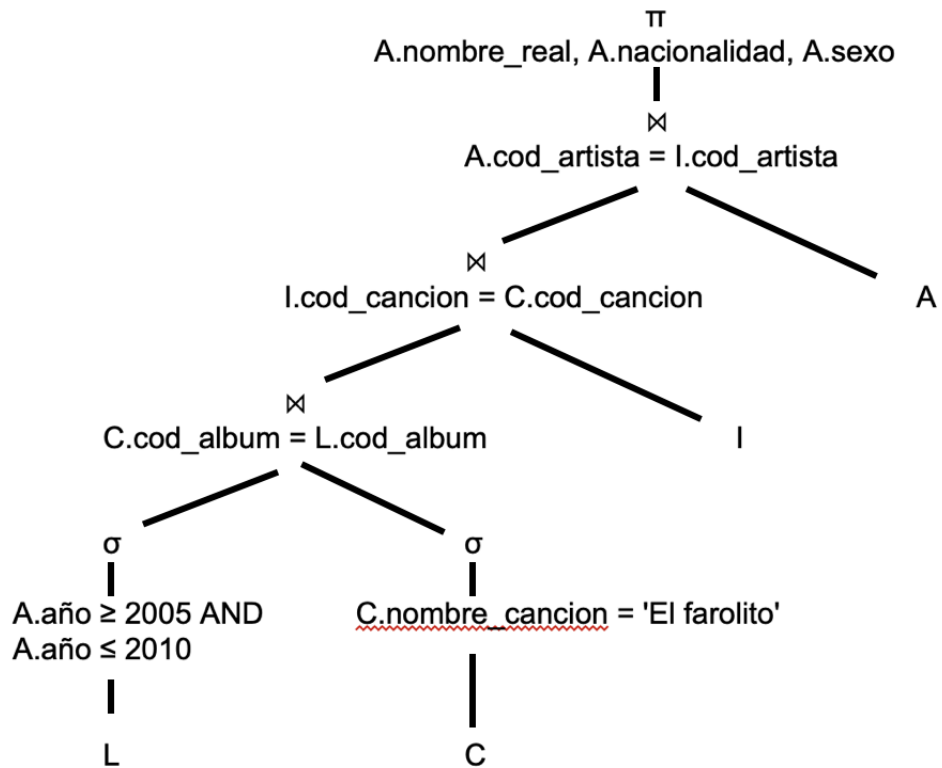
1) Aplicamos el Axioma 1 para descomponer el SELECT en una cascada de SELECT:



- 2) Desplazamos los SELECT hacia abajo del árbol, usando los Axiomas 2, 4, 6 y 10. Junto aplicamos los axiomas 5 y 9 para posicionar las relaciones más pequeñas en el hijo izquierdo.



3) Aplicamos el axioma 12 para la conversión del Producto Cartesiano en JOIN:



- 4) Aplicamos los Axiomas 3,4, 7 y 11 para bajar en el árbol las operaciones PROJECT lo más que se pueda y distribuir la proyección con respecto a la unión e intersección:

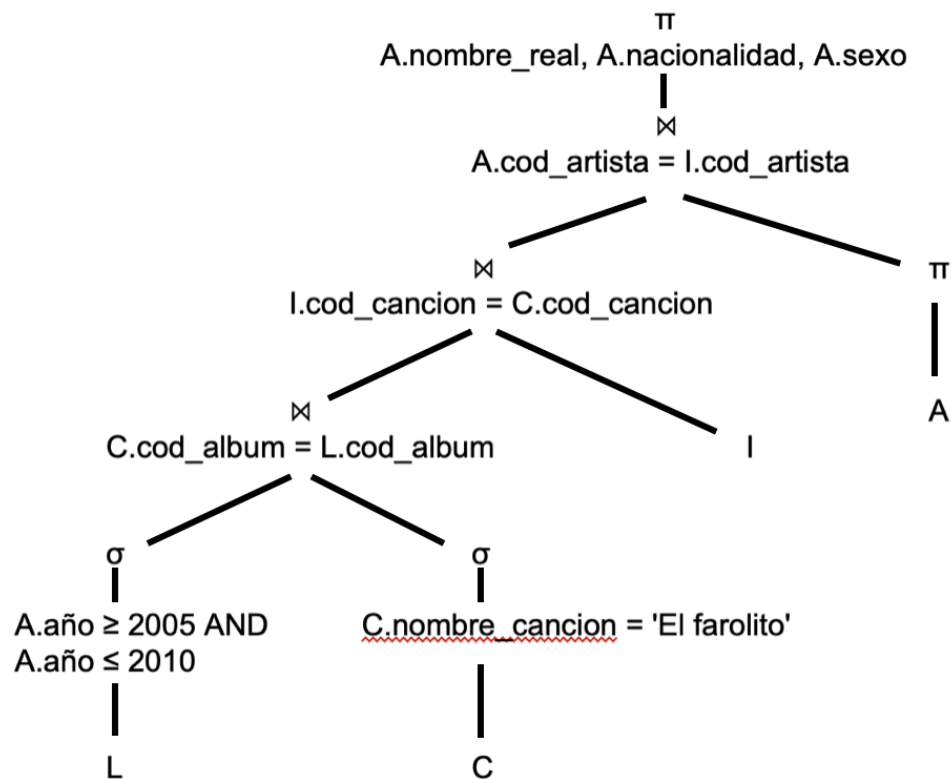


Imagen 2. Árbol optimizado empleando el método heurístico.

3. Calcule el tamaño de las tablas base

Las tablas base son las tablas que se ubican en los nodos hoja de árbol canónico. Estas tablas se obtienen del archivo de datos con **organización no extendida**. Corresponden a las tablas Artista, Género, Tiene, Interpreta, Canción, Álbum

Artista: Archivo de datos

cod_artista	nombre_real	nombre_artístico	nacionalidad	sexo	fecha_nacimiento
4	40	40	30	1	10

Tabla 4. Tamaño en bytes de los datos.

Nro de registros (nR): 10.000 (dato)

Tamaño del bloque (B): 2048 bytes (dato)

Tamaño del registro (R): $4 + 40 + 40 + 30 + 1 + 10 + 1$ (byte de borrado) = **126 bytes**

Factor de bloqueo (fb): $fb = \text{piso}(B/R) = \text{piso}(2048/126) = 16$

Tamaño del archivo (TA): $TA = \text{techo}(nR/fb) = \text{techo}(10.000/16) = 625$ bloques

Género: Archivo de datos

cod_género	nombre	descripción
4 bytes	40 bytes	50 bytes

Tabla 5. Tamaño en bytes de los datos.

Nro de registros (nR): 75 (dato)

Tamaño del bloque (B): 2048 bytes (dato)

Tamaño del registro (R): $4 + 40 + 50 + 1$ (byte de borrado) = **95 bytes**

Factor de bloqueo (fb): $fb = \text{piso}(B/R) = \text{piso}(2048/95) = 22$

Tamaño del archivo (TA): $TA = \text{techo}(nR/fb) = \text{techo}(75/22) = 4$ bloques

Tiene: Archivo de datos

cod_artista	cod_genero	fecha_inicio
4 bytes	4 bytes	10 bytes

Tabla 6. Tamaño en bytes de los datos.

Nro de registros (nR): 20.000 (dado)

Tamaño del bloque (B): 2048 bytes (dado)

Tamaño del registro (R): $4 + 4 + 10 + 1(\text{byte de borrado}) = 19 \text{ bytes}$

Factor de bloqueo (fb): $fb = \text{piso}(B/R) = \text{piso}(2048/19) = 108$

Tamaño del archivo (TA): $TA = \text{techo}(nR/fb) = \text{techo}(20.000/108) = 185 \text{ bloques}$

Interpreta: Archivo de datos

cod_artista	cod_cancion	tipo
4 bytes	4 bytes	10 bytes

Tabla 7. Tamaño en bytes de los datos.

Nro de registros (nR): 1.500.000 (dado)

Tamaño del bloque (B): 2048 bytes (dado)

Tamaño del registro (R): $4 + 4 + 10 + 1(\text{byte de borrado}) = 19 \text{ bytes}$

Factor de bloqueo (fb): $fb = \text{piso}(B/R) = \text{piso}(2048/19) = 108$

Tamaño del archivo (TA): $TA = \text{techo}(nR/fb) = \text{techo}(1.500.000/108) = 13889 \text{ bloques}$

Canción: Archivo de datos

cod_cancion	cod_album	cod_genero	nombre_cancion	duración
4 bytes	4 bytes	4 bytes	40 bytes	10 bytes

Tabla 8. Tamaño en bytes de los datos.

Nro de registros (nR): 500.000 (dato)

Tamaño del bloque (B): 2048 bytes (dato)

Tamaño del registro (R): $4 + 4 + 4 + 40 + 10 + 1(\text{byte de borrado}) = 63 \text{ bytes}$

Factor de bloqueo (fb): $fb = \text{piso}(B/R) = \text{piso}(2048/63) = 33$

Tamaño del archivo (TA): $TA = \text{techo}(nR/fb) = \text{techo}(500.000/33) = 15152 \text{ bloques}$

Álbum: Archivo de datos

cod_album	nombre	descripción	año	disquera
4 bytes	40 bytes	50 bytes	5 bytes	40 bytes

Tabla 9. Tamaño en bytes de los datos.

Nro de registros (nR): 6.000 (dato)

Tamaño del bloque (B): 2048 bytes (dato)

Tamaño del registro (R): $4 + 40 + 50 + 5 + 40 + 1(\text{byte de borrado}) = 140 \text{ bytes}$

Factor de bloqueo (fb): $fb = \text{piso}(B/R) = \text{piso}(2048/140) = 15$

Tamaño del archivo (TA): $TA = \text{techo}(nR/fb) = \text{techo}(6.000/15) = 400 \text{ bloques}$

4. Calcule el factor de selectividad de las selecciones y de los join. (2 pts)

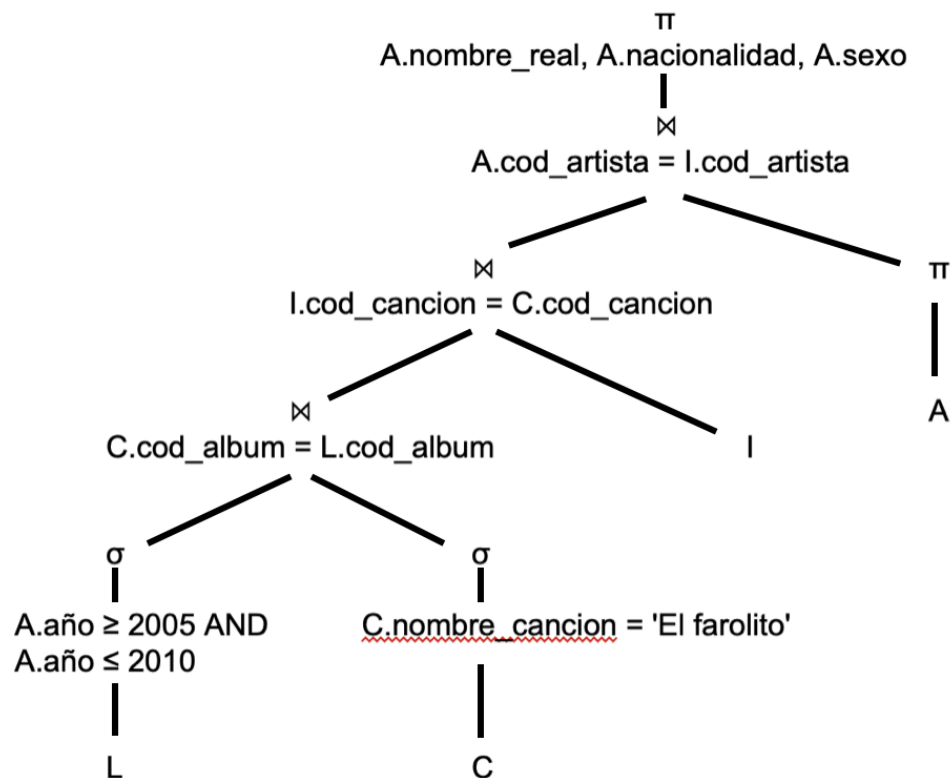


Imagen 3. Árbol optimizado heurísticamente.

- a) Factor de selectividad de $\sigma_{JC.nombre_cancion='El\ Farolito'}$: Como hay 1000 nombres de canción diferentes approx entonces los valores distintos es 1000.

$$f(JC.nombre_cancion='El\ Farolito',C) = 1 / |ValoresDistintos(nombre_cancion)| = 1/1.000 = \mathbf{0,001}$$

- b) Factor de selectividad de $\sigma_{JA.año \geq 2005 \text{ AND } A.año \leq 2010}$: Sabemos que el $\min(A.año) = 2005$ y que el $\max(A.año) = 2023$. Se hacen los factores de selectividad individualmente y luego se multiplican

- i) Factor de selectividad de $\sigma_{JA.año \geq 2005}$:

$$f(JA.año \geq 2005,A) = \max(A.año) - 2005 / \max(A.año) - \min(A.año) + 1 = 2023 - 2005 / 2023 - 2005 + 1 = 18/19$$

- ii) Factor de selectividad de $\sigma_{JA.año \leq 2010}$:

$$f(JA.año \leq 2010,A) = 2010 - \min(A.año) / \max(A.año) - \min(A.año) + 1 = 2010 - 2005 / 2023 - 2005 + 1 = 5/19$$

$$\text{Factor de selectividad} = 18/19 * 5/19 = \mathbf{0,249}$$

- c) Factor de selectividad de $\bowtie JC.cod_album = L.cod_album$: Como en la tabla Album hay 6.000 registros se asume que esa es la cantidad de valores distintos de cod_album en ambas tablas

$$f(JC.cod_album = L.cod_album, C) = 1 / |ValoresDistintos(cod_album)| = 1/6000 = \mathbf{0,00016}$$

- d) Factor de selectividad de $\bowtie JI.cod_cancion = C.cod_cancion$: Como en la tabla Cancion esta establecido que nada mas hay 1000 codigos de cancion approx. entonces se usa como la cantidad de valores distintos

$$f(JI.cod_cancion = C.cod_cancion) = 1 / |ValoresDistintos(cod_cancion)| = 1/1.000 = \mathbf{0,001}$$

- e) Factor de selectividad de $\bowtie JA.cod_artista = I.cod_artista$: Como en la tabla Artista hay 10.000 registros se asume que hay 10.000 artistas distintos y se toma ese valor ya que 1 artista interpreta varias canciones.

$$f(JA.cod_artista = I.cod_artista, A) = 1 / |ValoresDistintos(cod_cancion)| = 1/10.000 = \mathbf{0,0001}$$

En resumen, los factores de selectividad de cada SELECT y JOIN del árbol de ejecución optimizado son los siguientes:

Operación	Factor de selectividad
$\sigma_{JC.nombre_cancion='El\ Farolito'}$	$f(JC.nombre_cancion='El\ Farolito',C)=\mathbf{0,001}$
$\sigma_{JA.año \geq 2005 \text{ AND } A.año \leq 2010}$	$f(JA.año \geq 2005,A)*f(JA.año \leq 2010,A)=18/19 * 5/19 =\mathbf{0,249}$
$\bowtie_{JC.cod_album = L.cod_album}$	$f(JC.cod_album = L.cod_album,C)=\mathbf{0,000016}$
$\bowtie_{Jl.cod_cancion = C.cod_cancion}$	$f(JJl.cod_cancion = C.cod_cancion)=\mathbf{0,001}$
$\bowtie_{JA.cod_artista = l.cod_artista}$	$f(JA.cod_artista = l.cod_artista,A)=\mathbf{0,0001}$

Tabla 8. Factor de selectividad de las operaciones SELECT y JOIN.

